COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	**SECTION: CPE21S1**
DATE PERFORMED: Oct 6, 2026	**DATE SUBMITTED: Oct 6, 2026**
NAME: Mendoza, Roy Gabriel I. **Pilapil, Loreen Jae**	**INSTRUCTOR: Engr. Jimlord Quejado**

ACTIVITY NO. 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A:

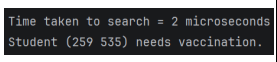
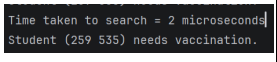
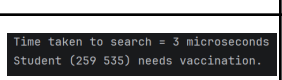
The total number of comparisons in the worst case is:

[n^2], since we assume that m is equal to n, we could just multiply it to each other, and since we follow the convention of n, we turn the m to n, so n times n, is equal to n^2.

Analysis of the Graph

If we are to follow the growth rate and the line in the graph, it would seem to be like an exponential growth, but based on the data, of O(n^2) meaning that it increases four times when the input size is doubled.

ILO B:

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	2 microseconds		The search happened almost instantly, for an input size of just 1000, the dataset is just minimal.
10000	2 microseconds		The same as the Input Size of 1000, with the same execution speed even though the input size increased ten times.
100000	3 microseconds		The dataset has increased by a 100 times compared to the initial dataset, but the execution size just added another microsecond. This shows the

			behavior of the binary search, where massive exponential increases in data yield only a tiny linear like increase in execution duration

CODE

```cpp
//
// Created by royga on 10/6/2026.
//
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;
public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() const ->pair<int, int> { return name; }
    auto is_vaccinated() const ->bool { return vaccinated; }

    // Two people are the same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
```

```
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

// Function to generate a student from random data
auto generate_random_Student(int max) ->Student {
    std::random_device rd;
    std::mt19937 rand( sd: rd());
    // The IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist( a: 1,  b: max);
    // Generate random credentials
    auto random_name :pair<unsigned, unsigned>  = std::make_pair( x: uniform_dist( [&] rand),  y: uniform_dist( [&] rand));
    bool is_vaccinated = uniform_dist( [&] rand) % 2 ? true : false;
    return Student( ∴ random_name, is_vaccinated);
}
```

```
// Function that implements the logic for whether a vaccination is needed using binary search
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true) {
        auto range_length :ptrdiff_t  = std::distance(first, last);
        auto mid_element_index :double  = std::floor( x: range_length / 2);
        auto mid_element :Student  = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance( [&] last, -mid_element_index);

        if (mid_element < P)
            std::advance( [&] first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}
```

```cpp
// Function to run and test the output using std::chrono for experimental analysis
void search_test(int size, Student p) {
    std::vector<Student> people;
    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second << ") needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second << ") does not need vaccination." << std::endl;
}
```

```cpp
// Driver code
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(max: 1000);
    search_test(size: 1000, p);
    search_test(size: 10000, p);
    search_test(size: 100000, p);
    return 0;
}
```

OUTPUT

```
"C:\Users\royga\CLionProjects\untitled1\HOA 11p1\HOA11.exe"
Time taken to search = 2 microseconds
Student (259 535) needs vaccination.
Time taken to search = 2 microseconds
Student (259 535) needs vaccination.
Time taken to search = 3 microseconds
Student (259 535) needs vaccination.
```

ANALYSIS

MENDOZA:
> The code uses a binary search algorithm operating on a sorted vector of student objects. The theoretical time complexity is)(log n) because the algorithm cuts the search in half during the iterations loops. And in the input of 1000, the maximum number of iterations

	required is log(1000), and so on just the n= changes depending on the amount of the input. Using the **std::chrono::steady_clock** , we can measure the amount of times it needed to execute every input size and loops, and we can see the differences of the three inputs on how long they needed to be executed.
	Pilapil: > The code implements binary search to look for a student in a sorted list. (log n) is the time complexity of binary search because it cuts the search area in half after each comparison. As we can see, the output supports this as the search time stays almost the same even as the number of samples increases.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

PROBLEM 1:

CODE

```cpp
// PROBLEM 1: Uniqueness Algorithm
bool Unique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false; // Duplicate found, not unique
            }
        }
    }
    return true; // All elements are unique
}
```

```cpp
int main() {
    //Problem 1: Unique
    std::cout << "PROBLEM 1: UNIQUE ALGORITHM" << std::endl;
    std::vector<int> test_array = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};

    auto start1 :time_point<steady_clock> = std::chrono::steady_clock::now();
    bool is_unique = Unique(test_array);
    auto end1 :time_point<steady_clock> = std::chrono::steady_clock::now();

    std::cout << "Array is unique? " << (is_unique ? "True" : "False") << std::endl;
    std::cout << "Time taken for Unique: "
              << std::chrono::duration_cast<std::chrono::microseconds>( d: end1 - start1).count()
              << " microseconds\n" << std::endl;
}
```

OUTPUT

```
PROBLEM 1: UNIQUE ALGORITHM
Array is unique? True
Time taken for Unique: 1 microseconds
```

THEORETICAL	The unique finder algo uses nested loops to see if the 2 loops(inner and outer) have matching pairs. And as for the time complexity, it has **O(n^2).**
EXPERIMENTAL	We have a 10 item array, which the program has completed in just 1 microseconds. Because the item size is small enough, the program finished executing the code in just a microsecond or almost instantly.

PROBLEM 2:

CODE

```
// PROBLEM 2: Recursive Power Functions
// Standard Linear Recursive Power: O(n)
long long rpower(long long x, int n) {
    if (n == 0) {
        return 1;
    } else {
        return x * rpower(x, n - 1);
    }
}
long long brpower(long long x, int n) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 != 0) { // If n is odd
        long long y = brpower(x, n: (n - 1) / 2);
        return x * y * y;
    } else { // If n is even
        long long y = brpower(x, n / 2);
        return y * y;
    }
}
```

```cpp
    //Testing Problem 2: rpower vs brpower
    std::cout << "PROBLEM 2: POWER FUNCTIONS" << std::endl;
    long long base = 2;
    int exponent = 20; // Using a safe exponent to prevent overflow for linear rpower

    // Test standard rpower
    auto start_r :time_point<steady_clock> = std::chrono::steady_clock::now();
    long long result_r = rpower(base, exponent);
    auto end_r :time_point<steady_clock> = std::chrono::steady_clock::now();

    std::cout << "rpower(" << base << ", " << exponent << ") = " << result_r << std::endl;
    std::cout << "Time taken by rpower: "
              << std::chrono::duration_cast<std::chrono::nanoseconds>( d: end_r - start_r).count()
              << " nanoseconds" << std::endl;

    // Test optimized brpower
    auto start_br :time_point<steady_clock> = std::chrono::steady_clock::now();
    long long result_br = brpower(base, exponent);
    auto end_br :time_point<steady_clock> = std::chrono::steady_clock::now();

    std::cout << "brpower(" << base << ", " << exponent << ") = " << result_br << std::endl;
    std::cout << "Time taken by brpower: "
              << std::chrono::duration_cast<std::chrono::nanoseconds>( d: end_br - start_br).count()
              << " nanoseconds" << std::endl;

    return 0;
}
```

OUTPUT

```
PROBLEM 2: POWER FUNCTIONS
rpower(2, 20) = 1048576
Time taken by rpower: 400 nanoseconds
brpower(2, 20) = 1048576
Time taken by brpower: 200 nanoseconds


Process finished with exit code 0
```

THEORETICAL	The standard rpower (x,n) functions decrements the exponent by 1 at each recursion until it hits the base number or case yielding a linear time complexity of O(n). In comparison to the brpower function that halves the exponent at every recursion through an odd and even branching, yielding a log time complexity of O(log n)
EXPERIMENTAL	The standard linear power function took 400 nanoseconds, in contrast the optimized binary power function only took 200 nanoseconds, half of the standard power functions, because it only did fewer recursion compared to the full steps of the rpower function.

ANALYSIS AND COMPARISON:

When comparing the two problems, it shows the differences between the quadratic and logarithmic growth behaviors in algorithms and time and space complexity. In Problem 1, the nested loop checks it quadratically, requiring n(n-1)/2 comparisons, but because of the tiny size of input of 10, the execution time is not that much affected by the size of the input, not showing any significant impact. Unlike the Problem 2, which highlights the divide and conquer optimization, for example, an exponent of n =20, the linear function took 400 nanoseconds, while the binary function, only took 200 nanoseconds, that is half the amount , showing that halving the size at each step cuts the recursions and produce a better runtime.

CONCLUSION:

Overall, the creation of this activity made us able to understand how to do Experimental and Theoretical Analysis techniques. From the procedure, we were able to see the basic differences or the effects of the high amount of input like a thousand or even a hundred thousand of random input, we were able to demonstrate that the binary search works well with large data sets.

Additionally, the supplementary activity made us able to understand the growth rate of nested loops with linear and binary power functions. The experimental code outputs confirmed that while inefficient algorithms can appear fast on tiny inputs due to modern hardware speeds and specs, their performance degrades rapidly as data scales or becomes larger. Ultimately, this activity emphasizes the importance of evaluating both theoretical behavior and experimental runtimes to design efficient, scalable software solutions that we can use and be able to apply in both our studies and possible work.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

